

CLAUDE_ANCHORS_FULL

§11.4.48 — UI-driven video testing mandate (User mandate, 2026-05-18)

Every test that asserts video playback on a secondary display MUST traverse the user-equivalent UI path (launcher icon → app home → content list → tile tap → playback → in-app pause/resume → back button stop). NOT Intent/Broadcast shortcuts (`am start -a VIEW` , `cmd media_session play`). Real `uiautomator dump` element resolution

- `input tap X Y` . Coverage: every video-capable app in `PRODUCT_PACKAGES` + every stream type (progressive HTTP / HLS / DASH / RTMP / file-local / DRM) + every codec (H.264/265/VP9/AV1/MPEG-2/MP4V video; AC-3/E-AC-3/TrueHD/DTS/DTS-HD/MLP/Opus/AAC audio). Secondary display verified via `ffprobe-on-captured-mp4` + VOM `activeDecoder` state. Arvus codec-state cross-check per §11.4.13 + §CG screenshot.

Per §11.4.4 four-layer: pre-build gate `CM-AF-UI-DRIVEN-VIDEO- COVERAGE` + propagation gate `CM-COVENANT-114-48-PROPAGATION` + on-device test framework at `device/rockchip/rk3588/tests/ui_driven/` (Layer 1 helper + Layer 2 per-app drivers + Layer 3 scenarios) + Layer 4 orchestrator `scripts/testing/run_ui_driven_video_suite.sh` + paired meta-test mutations.

Carve-out (User mandate 2026-05-20). The 5 canonical tracker documents — `docs/Issues.md` , `docs/Issues_Summary.md` , `docs/Fixed.md` , `docs/Fixed_Summary.md` , `docs/CONTINUATION.md` — sit at `docs/` root by design. They are architectural constants of the project layout, analogous to AOSP's `Makefile` , `Android.bp` , `OWNERS` files at repo root. Their location is encoded as literal path strings in §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.44 + §11.4.53 propagation gates plus the helper-script constellation that regenerates them. Moving them would require coordinated amendment of those 6 sister anchors plus 5 pre-build gates plus ~20 helper scripts plus 42 consumer files in a single PWU. Per §11.4.66, that scope is operator-blocked until explicitly authorised. Audit-snapshot files (`docs/audit/anti_bluff_audit.md` , `docs/audit/`

`PRE_SONOS_TAG_READINESS.md` ,
`docs/audit/D1_WIFI_FAIL_CLASSIFICATION.md` , plus any future audit snapshots) DO move under `docs/audit/` per the §11.4.11 general principle.

Canonical authority: constitution submodule `Constitution.md` §11.4.48.

Non-compliance is a release blocker regardless of context.

§11.4.49 — Dual-approach testing mandate (User mandate, 2026-05-18)

Every feature test exercising a user-visible behaviour MUST ship in TWO variants: a UI-driven variant (uiautomator-based, §11.4.48 surfaces A–E) AND an Intent/Broadcast-driven variant (`am start --es` / `am broadcast` -based). Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack — UI catches app-side bugs; Intent catches framework/system-server bugs.

Shared assertion base at `tests/lib/dual_approach_test_base.sh` :

`dat_init` / `dat_start_capture` / `dat_assert_codec_state` /
`dat_assert_video_frames` / `dat_assert_audio_channels` /
`dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_finding` . Both variants gather identical evidence into mirror directories `qa-results/dual_approach/``<F>``<run-ts>``{ui,intent}/` so the orchestrator diffs results and pinpoints which half of the stack contains a bug.

Kinopoisk 5.1 EAC3 is the canonical first implementation. Both variants are RED per §11.4.43 until the §CN decoder pipeline fix lands.

Pre-build gates: `CM-COVENANT-114-49-PROPAGATION` (anchor across parent + 42 consumer files) + `CM-AF-DUAL-APPROACH-COVERAGE` (shared base contract) + `CM-AF-KINOPOISK-5-1-DUAL-COVERAGE` (both variants exist + share the base). Three paired meta-test mutations.

No escape hatch — no `--ui-only` / `--intent-only` / `--skip-dual` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.49.

Non-compliance is a release blocker regardless of context.

§11.4.50 — Deterministic consistency mandate (User mandate, 2026-05-18)

Every test that PASSes MUST have been executed N times (default N=3 normal tests, N=10 cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration. A divergent N-iter run is auto-FAIL — there is no "first PASSED therefore X was a flake" path.

§11.4.7's `intermittent` / `transient` / `flake` / `flaky` vocabulary is enforced MECHANICALLY by this mandate, not merely textually.

Coverage: every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥ 1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit; the threshold ratchets 70 $\rightarrow$ 85 $\rightarrow$ 95 $\rightarrow$ 99 over phases.

Reliability mechanism: `ab_run_n_times <test_name> <N> <fn> [args...]` helper in the project anti-bluff library loops, captures evidence-hash per iter, asserts all N hashes + exit codes identical. NO operator-facing escape converts divergence to PASS.

Pre-build gates: `CM-COVENANT-114-50-PROPAGATION` + `CM-AF-RELIABILITY-CHECK-WIRED` + `CM-AF-FEATURE-COVERAGE-MATRIX` . Three paired meta-test mutations. No escape hatch — no `--allow-flake` , `--first-pass-suffices` , `--skip-n-iter` , `--skip-coverage-audit` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.50.

Non-compliance is a release blocker regardless of context.

§11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18)

Every fix MUST be classified by rebuild-requirement before commit using the project's per-file-class decision matrix. If `LIVE_ADB_TESTABLE` (on-device test scripts, host scripts, atmosphere- *.sh boot scripts, persist.* properties, markdown docs, test fixture assets), the operator MUST first apply the fix to the running device via `adb push` / `setprop` / `pm install -r` / `mount -o remount,rw` , run the §11.4.43 RED test live, capture PASS, THEN commit + rebuild + reflash as belt-and-suspenders re-validation. Commit footer: `LIVE_ADB_VALIDATED: yes` . If `REQUIRES_REBUILD` (kernel, framework Java/AIDL, native C++ in APEX, sepolicy,

init.rc, ro.* properties, XML overlays, codec XML in APEX, Android.bp/.mk), the operator proceeds directly to source-side + rebuild. Commit footer:

```
REQUIRES_REBUILD: <reason> . Mixed batches use partial.
```

§11.4.51 REFINES §11.4.43 step 2 with mechanical enforcement. Helper:

```
scripts/testing/classify_fix_rebuild_requirement.sh walks  
git diff --name-only , looks up each file against the matrix, emits per-file  
classification + recommended commit-message footer. Unmatched paths classify  
as REQUIRES_REBUILD: unmatched-path (safe default per §11.4.6). Pre-build  
gates: CM-COVENANT-114-51-PROPAGATION + CM-AF-CLASSIFY-FIX-HELPER-  
EXISTS
```

- CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER . Three paired meta-test mutations.
No escape hatch — no --skip-classify / --assume-rebuild / --no-
footer-required flag.

Canonical authority: constitution submodule Constitution.md §11.4.51.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

§11.4.54 — ATM-NNN ticket identifier mandate (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

§11.4.55 — Reopens-history tracking + per-item Reopens.md doc (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

§11.4.56 — Status_Summary parity + two-audience format (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

§11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19T~05:00Z MSK):

§11.4.59 — README always-sync mandate (User mandate, 2026-05-19)

`README.md` is a §11.4.12-class always-sync document: HTML + PDF exports refresh on every update via `scripts/testing/sync_readme_export.sh` (pandoc + weasyprint); auto-invoked by `sync_issues_docs.sh` so a single doc-sync run refreshes Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README (md + html + pdf). README carries a §11.4.44 revision header and a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation. Pre-build gate

`CM-README-EXPORT-SYNC` enforces mtime parity (README.html + README.pdf ≥ README.md). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No

escape hatch — no `--skip-readme-sync` , `--no-readme-export` , `--readme-stale-ok` flag. Composes with §11.4.12 + §11.4.18 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.59.

Non-compliance is a release blocker regardless of context.

§11.4.60 — Documentation always-sync composite covenant (User mandate, 2026-05-19)

Eight documentation classes (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md, every Status_Summary.md) MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` that FAILs if ANY instance's `.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/**` recursively for Status.md fleet. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch — no `--skip-composite-doc-sync` , `--allow-stale-html` , `--summary-not-applicable` flag exists. Composes with §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.23 + §11.4.33 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §11.4.59 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.60.

Non-compliance is a release blocker regardless of context.

§11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19)

Every workable-item action (open / update / close / reopen / migrate) MUST follow the canonical procedure document at `docs/procedures/issues/<Action>.md` . Closed-set of 5 procedure docs: `Creation.md` , `Updating.md` , `Resolution.md` , `Reopening.md` , `Migration.md` . Each carries §11.4.44 revision header + HTML + PDF exports (refreshed by `scripts/testing/sync_procedure_docs_export.sh` ,

invoked as the final stage of `sync_issues_docs.sh`). ALL work — new features, behavioural changes, tasks (refactor / doc / infra / gate / audit / cleanup), bug fixes, investigation, documentation — MUST flow through these procedures. No ad-hoc procedure permitted.

Pre-build gate `CM-PROCEDURES-DOCS-PRESENT` checks all 5 procedure docs exist + carry revision header + have current HTML/PDF exports + sync helper is wired into `sync_issues_docs.sh` . Paired mutation: rename one procedure doc → gate FAILs. Propagation gate `CM-COVENANT-114-63-PROPAGATION` enforces this anchor literal in every `CLAUDE.md` / `AGENTS.md`.

Canonical authority: constitution submodule `Constitution.md` §11.4.63.

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md` , `docs/**/*.*md` , `scripts/**/*.*md` (doc-format companion docs), owned-submodule top-level `README.md` / `CLAUDE.md` / `AGENTS.md` / `CHANGELOG.md` and their `docs/**/*.*md` , `constitution/**/*.*md` , owned HelixQA submodules' equivalents. Excludes: `external/**` , `prebuilts/**` , `packages/modules/**` , `kernel-5.10/**` , `out/**` , `build/**` , application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be $\geq$ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION` . Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports` , `--no-pdf-only` , `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask` , `--silent-wait` , `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!"

- "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh

parses the whole script BEFORE executing, so the runtime `[ -n "$ {BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee ...) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules).

OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20)

A test asserting audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only. Closed enumeration of acceptable evidence: (1) sink-side codec-state showing non-empty codec matching the expected regex during playback (Arvus / Sonos / Yamaha REST API); (2) PCM `hw_ptr` delta strictly positive across the playback window; (3) ALSA ELD demonstrating negotiated channel count + format; (4) `ffprobe-on-captured-mp4` with non-zero frames matching expected codec; (5) recording-analyzer matched event per §11.4.2 / §11.4.5 timeline; (6) tinycap RMS amplitude above floor.

Failure modes specifically forbidden: `arvus_probe_present` returning 1 silently dropping the test to SKIP; empty codec-state treated as evidence; `umpsys media.audio_flinger` policy-side device field used as proof PCM actually opened; config-XML-only PASS without runtime evidence. All four are the §11.4 PASS-bluff pattern materialised on D3 2026-05-20 when the test suite reported audio- routing PASS while the user heard nothing and Arvus Codec-In-Use was empty.

Mandatory protections: (1) sink-side library helpers expose

`*_require_reachable` returning exit code 2 (OPERATOR-BLOCKED) when sink unreachable; (2) test harness propagates 2 to summary as OPERATOR-BLOCKED, release tags blocked while OPERATOR-BLOCKED exists; (3) audio-routing tests rewritten to capture ≥ 1 positive sink/downstream evidence per the enumeration above; (4) anti-stickiness post-stop — test re-probes sink and asserts codec-state transitioned to N.E. / 0-frames-delta. No escape hatch — no `--skip-sink-evidence` , `--allow-empty-codec` , `--sink-unreachable-is-pass` , `--metadata-only-suffices` flag.

Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.14 / §11.4.46 / §11.4.49 / §11.4.50 / §11.4.52. Pre-build gates `CM-COVENANT-114-68-PROPAGATION` + `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` with paired meta-test mutations per §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

§11.4.70 — Subagent-driven execution is the default (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20)

Everyday-push variant of §11.4.41 (force-push merge-first). Before pushing to any upstream for any repository (main repo or submodule), the agent MUST follow the 5-step pre-push cycle: (1) `git fetch --all --prune --tags`, (2) `git pull --no-rebase <remote> <branch>` for each remote whose tip differs from local, (3) investigate the diff vs OUR previous HEAD by reading every foreign commit's body (what changed, why, how it affects OUR project), (4) integrate mandatory changes with full §11.4.4(b) four-layer anti-bluff test coverage producing REAL captured-evidence proofs, (5) THEN push to every configured remote in cascade order.

Composes with §11.4.41 (force-push case) + §11.4.26 / §11.4.32 / §11.4.37 / §11.4.40 / §11.4.42 / §11.4.43 / §11.4.4(b) / §11.4.5 / §11.4.6.

Applies to parent repo + constitution submodule + every owned submodule + every nested submodule + every HelixQA dependency. Audit-trail per push reconstructable from `docs/changelogs/<tag>.md` + per-repo `git log` evidence.

No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. Pre-build gate `CM-COVENANT-114-71-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make sure all fixes for audio are always top priority in main working stream!" The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Audio scope includes §EU HDMI rejection, §EM multichannel HDMI, §ET Arvus integration, §EV/§EW/§EX D3 audio defects, HiFi (Fix #74, #76), AC3 (Fix #106), ES8388 (Fix #103/§104/#105), multichannel LPCM (Fix #112), and every future audio-stack improvement.

Composes with §11.4.42 (iteration-discipline — audio sits at apex of priority order) + §11.4.58 (parallel-development PWU — background research subagents run concurrently but do NOT preempt audio on the main-stream serial dispatch queue).

Operative rule: any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. No escape hatch — no "but this non-audio task is faster" or "but this research is more interesting" override. Audio-stack regressions are user-perceptible and high-impact (D3 silent post-flash is a release blocker), while research and refactors can wait.

Canonical authority: constitution submodule `Constitution.md` §11.4.72.

Non-compliance is a process violation regardless of context.

§11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!"

Applies only when a project recognises a main specification document (e.g. `docs/specs/**/specification.V<n>.md`). Two-axis versioning: **primary** (V1/V2/V3/...) bumps for major rewrites (old primary archived to `<spec-dir>/archive/`); **secondary** (`Revision`) bumps for additive operator requirements, refinements, polish (matches the §11.4.61 metadata-table `Revision` integer). Every operator-mandated requirement MUST land in the spec as part of the work that implements it. Cross-doc propagation copies MUST reference the active spec file, not a stale archived version. Composes with §11.4.44, §11.4.61, §11.4.59, §11.4.65.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

Non-compliance is a release blocker.

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20)

Direct user mandate (verbatim): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the contributor (human or AI agent) MUST: (1) survey the `vasic-digital` + `HelixDevelopment` orgs on GitHub + GitLab — the canonical inventory is `submodules-catalogue.md` (142 repos categorised); (2) reuse an existing Submodule when it covers $\geq 80\%$ of the functionality; (3) extend in-place via upstream PR when $80\%+$ matches but features are missing — never duplicate the 80% in-project; (4) document the survey result in the relevant tracker row with a `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` line.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

Canonical authority: constitution submodule `Constitution.md` §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

§11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and

unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"

The §11.4 covenant historically relied on agent + operator vigilance. Three forensic incidents in 2026-05-19 → 20 (two stalled remediation subagents + post-§11.4.66 propagation drift requiring catch-up commit `f93b25a92eb`) demonstrated that late-binding enforcement at `pre_build_verification.sh` time fires hours-to-days AFTER the violator commit has already reached every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline:

1. **Local pre-commit git hook** — refuses staged `.md` lacking sibling `.html` + `.pdf` (and other staged-only invariants).
2. **commit_all.sh integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before commit (`_constitution_sibling_check` function).
3. **Local pre-push git hook** — re-runs siblings + propagation-gate subset on every commit in the push.
4. **post-commit auto-repair hook** — detects orphan `.md` in just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **Local-only equivalent** (Phase 39.GF, User mandate 2026-05-20). Remote CI surfaces (GitHub Actions, GitLab pipelines, Jenkins, CircleCI, etc.) are DISABLED — the workflow file is preserved at `.github/workflows/constitution-compliance.yml.disabled-local-only` (NOT `.yml` ; GitHub Actions ignores it). Layer 5 enforcement migrated to the LOCAL pre-build-verification + meta-test ritual the operator MUST execute before tagging per §11.4.40. Layers 1-4 remain authoritative; Layer 5 is the operator's local final gate. A future re-enable PWU may re-establish remote CI.

Helper contracts (mandatory): `scripts/install_git_hooks.sh` (idempotent installer wired into `scripts/setup.sh`), `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}` , `_constitution_sibling_check` in `scripts/commit_all.sh` . The `commit-msg` hook enforces a `Bypass-rationale`:

<reason> footer when `--no-verify` is detected (touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`); `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Five pre-build gates with paired §1.1 meta-test mutations: `CM-COVENANT-114-75-PROPAGATION` (anchor literal across canonical files), `CM-GIT-HOOKS-INSTALL-SCRIPT` (installer present + executable), `CM-GIT-HOOKS-SOURCE-DIR` (4 hook bodies present + executable), `CM-COMMIT-ALL-SIBLING-CHECK` (`_constitution_sibling_check` in `commit_all.sh`), `CM-CI-WORKFLOW-PRESENT` (CI workflow + ≥ 3 required jobs).

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI makes the §11.4 promise mechanical), §11.4.41 (this anchor's renumber from §11.4.74 → §11.4.75 was itself driven by §11.4.41 merge-first discipline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive clarification), §11.4.67 (target-shell-parseability — hooks parse under bash AND mksh), §11.4.71 (pre-push fetch + integrate — Layer 3 + 5 honour the merge-first pipeline), §11.4.72 (audio top-priority — hooks hold no lock themselves, so do not race against in-flight audio commits), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch — no `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context.

§11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly:

`https://github.com/vasic-digital/containers` (`git@github.com:vasic-`

`digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

§-slot history note. Originally drafted as §11.4.75, renumbered to §11.4.76 per §11.4.71 fetch-before-push after the concurrent `0a70083` landing of §11.4.75 (Mechanical Enforcement). Same collision-resolution pattern as §11.4.75's own renumber from drafted §11.4.74.

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via `pkg/boot` + `pkg/compose` + `pkg/health` so operators are never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (the **on-demand-infra invariant**); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Composes with §11.4.74 (catalogue-first), §11.4.75 (mechanical enforcement — this clause IS one of the things mechanical enforcement protects), §3 (propagation), §11.4.31 (Submodule-Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase), §1.1 (paired-mutation gates protect the Submodule's own primitives).

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!"

Forensic anchor. 2026-05-20T15:00Z: a consuming project's audio Tier 1

`commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` — all untracked but un-gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing `RKTools`, missing test infra, missing build outputs).

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (download from authoritative source: vendor tarball, SDK installer, npm / pip / cargo / go-mod / container registry, dedicated git submodule, S3/GCS) OR **re-generate** (run from tracked source code via build pipeline, code-gen, asset render, captured-evidence replay, container build, kernel build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) entry in `scripts/setup.sh` (post-clone bootstrap) that runs the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok` recent; (4) README + relevant `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are themselves a §11.4 PASS-bluff variant — codebase appears complete to casual eye but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling + verifies `script-path` exists + parses YAML + checks bootstrap reference. Paired §1.1 mutation strips one required YAML key → gate FAILs.

Composes with §11.4.6 (no-guessing — mechanism MUST be verified working on sandbox clone, not assumed), §11.4.65 (universal Markdown export — generated HTML/PDF siblings are a `re-generate` instance), §11.4.66 (interactive clarification — ASK operator if mechanism unknown), §11.4.71 (pre-push fetch + integrate — re-validate vendor integrity before push), §11.4.74 (catalogue-first — extend a reusable downloader Submodule rather than reimplement), §11.4.75 (Mechanical Enforcement — pre-commit + CI replay layers refuse bare additions), §11.4.76 (Containers-submodule — container images regenerated via `vasic-digital/containers`), §9 / §9.2 (zero-risk data safety — pre-test mechanism in sandbox with hardlinked-backup), §3 (propagation — consuming submodules inherit §11.4.77).

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph> , npm `@colbymchenry/codegraph`) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud, no external API). (1) Install globally via npm — no `sudo` (the npm prefix MUST be user-writable). (2) `codegraph init` + `codegraph index` : `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude — non-negotiably — every credential/secret path per §11.4.10 (a secret reaching the index is a §11.4.10 violation). **CRITICAL — CodeGraph filter mechanism:** CodeGraph v1.x is zero-config. It does NOT use `config.json` `include` / `exclude` fields for file discovery. Filtering is done by a built-in ignore matcher seeded with defaults (`node_modules` , `vendor` , `dist` , `build` , `target` , `.venv` , `Pods` , `.next`) PLUS the project root `.gitignore` . To control indexing, use root-anchored `.gitignore` patterns with `!` negations for re-inclusion under default-skipped

parents (e.g., `!/vendor/widevine_*` re-includes owned vendor code). **ALL SOURCE CODE MUST BE INDEXABLE** — including AOSP platform trees (`frameworks/`, `packages/`, `external/`, `art/`, `bionic/`, `cts/`, `kernel-5.10/`, `hardware/*/`, `vendor/`, `device/*/`). Only actual rubbish MAY be excluded: build outputs (`/out/`, `/prebuilts/`, `/rockdev/`), caches (`.gradle/`, `__pycache__/`), credentials/secrets (`/secrets/`, `/qa-results/`, `/recordings/`). A forked-AOSP project with 1M+ tracked files will produce a correspondingly large index (30-40 GB for first init), which is expected and correct — the index must represent the full codebase the AI agent cross-references. (3) Wire the `codegraph serve --mcp` server into every CLI agent the developers use — project-scoped + committed where supported (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`), host-local otherwise (Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH` (no hardcoded host path). (4) Cover the integration with an anti-bluff verification suite whose per-agent end-to-end layer uses an **unforgeable challenge** — a fact obtainable only by calling a CodeGraph MCP tool (e.g. the index node count via `codegraph_status`) so an agent answering from its own file-reading tools cannot produce a false PASS; an agent that genuinely cannot be driven end-to-end (missing credentials / quota / environment incompatibility) is a documented SKIP gap per §11.4.3, never a faked PASS. (5) Document everything in `docs/CODEGRAPH.md`, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not added as a git submodule, and it adds no Git remote.

Universal mandatory-exclude baseline set (§11.4.78 + §11.4.79 refinement — binds EVERY consuming project). Independently of project, the CodeGraph index MUST NEVER include the following file-classes anywhere — they are indexed-rubbish in every project; a consuming project MAY add its own project-specific rubbish paths on top per §11.4.35 but MUST NOT drop any baseline class: (a) **build outputs / regenerable artifacts** — `/out/`, `/build/`, `/dist/`, `/target/`, `bin/`, `obj/`, `.compressed/`, and any generator-produced tree; (b) **caches / dependency / tooling dirs** — `**/ccache/`, `**/.ccache/`, `.gradle/`, `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.terraform/`, `.venv/`, `logs/`; (c) **credentials / secrets** — `.env`, `.env.*`, `**/*.env`, `secrets/`, `**/*secret*`, `keystores` / `signing-keys` / `service-account JSON`, and every

credential-bearing path per §11.4.10 (a secret reaching the index is a §11.4.10 violation, never negotiable); (d) **QA / recording / large regenerable corpora** — `**/qa-results/` , `**/recordings/` , and prebuilt-binary trees. Realized via root-anchored `.gitignore` patterns (CodeGraph v1.x zero-config filter = built-in skip list + root `.gitignore` , NOT `config.json` include/exclude). Own-org submodule sources stay INCLUDED per §11.4.79; third-party vendored trees EXCLUDED. This baseline is UNIVERSAL (§11.4.17) — dropping any class is a §11.4.78 / §11.4.79 violation, and no source-code tree may be excluded to satisfy it.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4 (anti-bluff — CI green is necessary, never sufficient), §1.1. Planned gate `CM-CODEGRAPH-WIRED` + paired §1.1 mutation (strip a secret-exclusion from `config.json` → gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach of this mandate.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

Refines §11.4.78 step 2's exclude-list with a per-submodule-ownership split: (a) **Own-org submodules** — full write access via the project's CLIs (canonical orgs: `vasic-digital` on GitHub/GitLab/GitFlic/GitVerse + `HelixDevelopment` on GitHub) — MUST be **INCLUDED** in the index. (b) **Third-party submodules** (the §11.4.74 `no-match` → `vendor` path, e.g. `gopkg.in/telebot.v3`) — MUST be **EXCLUDED**. Operational steps: (1) `git submodule update --remote --merge` to pull latest from every submodule before re-indexing — respect load-bearing pins on third-party submodules (e.g. roll back if `--remote` advances `gopkg.in/telebot.v3` past the v3 import-path pin). (2) Adjust `.codegraph/config.json` exclude list to keep own-org paths in scope. (3) Re-index via `scripts/`

`codegraph_setup.sh` . (4) Verify via `scripts/codegraph_validate.sh` with at least one probe that resolves a symbol living ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore.

Composes with §11.4.74 (catalogue ownership classifier), §11.4.78 (CodeGraph parent mandate), §11.4.10 (credentials still excluded regardless), §1.1 (paired mutation), §107 (an index that lies about reachable symbols is a §107 PASS-bluff against AI agents).

Canonical authority: constitution submodule `Constitution.md` §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded WITHOUT an audit trail in `.codegraph/config.json` comments) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! [...] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule since they MUST BE available to all projects which do respect and follow our constitution rules and mandatory constraints! Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents (another area / context to regularly update and sync) and regularly export them like all other Status docs into the PDF and HTML!"

Three deliverables (all in this constitution submodule): (1) `scripts/codegraph_update.sh` — npm-installs latest `@colbymchenry/codegraph` after npm-registry version check; appends old/new version to `docs/codegraph/Status.md` ; §107 anti-bluff: verifies `codegraph --version` reflects the new version after install (npm exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update, runs `codegraph status` → `codegraph sync` . → `codegraph status` → project's `scripts/`

`codegraph_validate.sh` in the consuming project; appends every step's output to BOTH the project's and the constitution's `docs/codegraph/Status.md` . (3) `docs/codegraph/Status.md` + `Status_Summary.md` append-only ledgers tracking all CodeGraph-related events; exported to `.html` + `.pdf` siblings per §11.4.65.

Operational cadence: weekly floor (per §11.4.45 status-digest cadence). Scripts are inherited by reference (per §3 submodule inheritance) — consuming projects invoke them at `${CONST_DIR}/scripts/codegraph_*.sh` , never copy.

Composes with §11.4.78 (CodeGraph parent), §11.4.79 (own-org submodule inclusion), §11.4.10 (credentials excluded), §11.4.45 (status-digest cadence), §11.4.53 (Fixed_Summary backfill), §11.4.65 (multi-format export), §107 (observed-version evidence per update; observed PASS/FAIL per sync), §1.1 (paired mutation: downgrade installed version → script detects drift → restore).

Canonical authority: constitution submodule `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (consuming project has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation that depends on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED.** Linux `cgroup/systemd/ /proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit / ulimit` , macOS `launchd` , BSD `rctl` , Windows Job Object) chosen via runtime dispatch — same operator-path invocation, OS-correct mechanism. **(B) Per-OS tests REQUIRED.** Every gate test that exercises platform-dependent behaviour MUST have `case "$(uname -s)" in` branches with positive captured evidence per

§11.4.2 + §11.4.5 in each branch. SKIP-with-reason is acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED.** Where a Linux primitive has NO macOS/ other-OS equivalent due to a documented kernel limitation (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + link to honest-gap doc, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. `RLIMIT_CPU` + `SIGXCPU` as the macOS proxy for "process is bounded"). The adjacent test MUST itself be anti-bluff with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical, not exhaustive): `systemd-run --user --scope` ↔ POSIX `ulimit -t -u` / `launchd`; `cgroup MemoryMax` ↔ XNU gap (use `RLIMIT_CPU` adjacent) / `rctl` / Job Object; `cgroup TasksMax` ↔ `RLIMIT_NPROC` ; `/proc/<pid>/oom_score_adj` ↔ no equivalent on Darwin/BSD (no kernel OOM-killer).

Composes with §11.4.1 (FAIL-bluffs forbidden — Linux-PASS / Darwin-SKIP without honest reason is a §11.4.81 violation), §11.4.2 (recorded evidence per branch), §11.4.3 (§11.4.81 strictens §11.4.3: topology SKIP only when kernel CANNOT, not when "we didn't implement yet"), §11.4.4 (test-interrupt triggers when discovering a Linux-only test without per-OS equivalent), §11.4.5 (captured evidence per platform), §11.4.6 (no guessing about platform capability — prove via reproducer), §11.4.20 / §11.4.70 (per-OS branches are parallel-subagent dispatchable), §11.4.27 (100% test-type coverage applies per-platform), §11.4.69 (sink-side evidence per platform), §107 (multi-platform projects must guarantee end-user usability on every platform). Pre-build gate `CM-CROSS-PLATFORM-PARITY` (when implemented): scans every gate test for `case "$(uname -s)"` blocks, asserts a non-SKIP branch (or honest-gap citation per (C)) exists for each platform in the supported-platforms manifest. Paired §1.1 mutation: strip a Darwin branch → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects. No escape hatch.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Forensic anchor — direct user mandate (verbatim, 2026-05-22): "How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!"

Iteration cycle time is a first-order quality enabler. Slow cycles bound the project's defect-discovery rate; fast cycles multiply it. The 2026-05-22 forensic session witnessed ~3 hours of operator wait time on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following speedup disciplines AS MANDATORY (each independently enforceable):

- **(A) Phase 1 forensic before any speculative source patch** — before any non-trivial source patch, complete `superpowers:systematic-debugging` Phase 1 to identify FACT-grade root cause. Speculative patches without Phase 1 evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any rebuild** — strengthens §11.4.51 to a release-blocker mandate. Skipping live-probe for a `LIVE_ADB_TESTABLE` change is a §11.4.82 violation.
- **(C) Pre-flight before launching rebuild orchestrators** — 30-second pre-flight verifies device reachability, sink reachability, host memory/disk, no stale locks, no orphan processes. A rebuild against a broken precondition wastes 45 min.
- **(D) Persistent build caches outside containers** — `ccache` + Soong / `sccache` / Gradle daemon state bind-mounted to host, NOT in ephemeral container layer. Cold rebuild ~40 min → warm rebuild 5-15 min.
- **(E) Module-only rebuild for loadable-module-only changes** — `make -C kernel M=<driver-path> modules` for `CONFIG_*=m` drivers (~2-5 min). Full rebuild required only for `=y` built-ins.
- **(F) Parallel multi-device testing** — every owned validation device runs the autonomous cycle concurrently with separate `qa-results/<TS>/<device-tag>/` outputs.
- **(G) Subagent scope discipline + worktree isolation** — subagents ≤30 min budget, single-responsibility, intermediate output emitted as they progress, `isolation: "worktree"` by default.

- **(H) Lock-file + stale-process hygiene** — detect and clean `.git/`
`index.lock` , `.lock` files, orphan `git /build` processes on session start.
Disable auto git-gc (`gc.auto 0`) in concurrent multi-agent repos.
- **(I) Cycle telemetry per §11.4.24** — every iteration logs commit hash, per-phase wall-clock, speedup-discipline flag set, outcome. Aggregated weekly to surface which disciplines deliver the biggest empirical return.

Composes with §11.4.4, §11.4.6, §11.4.9, §11.4.20 / §11.4.70, §11.4.24, §11.4.42, §11.4.43, §11.4.50, §11.4.51, §11.4.52, §11.4.58, §12.7, §107. Pre-build gate `CM-ITERATION-SPEEDUP-DISCIPLINE` (when implemented): audits the most recent N cycles for telemetry citing which of (A)-(I) applied. Paired §1.1 mutation: strip the speedup-flag column → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.82.

Non-compliance is a release blocker. No escape hatch — no `--skip-phase1-forensic` , `--no-pre-flight` , `--rebuild-everything-always` , `--unlimited-subagent-scope` , `--ignore-locks` , `--no-telemetry` flag exists.

§11.4.100 — RETIRED. Demoted to consumer project (a consuming project's video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, 2026-06-08). Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a registered guard is a §11.4.123 violation. The suite runs FIRST in the post-deploy cycle (highest-risk set

per §11.4.132) and is a §11.4.40 release-gate blocker. Forensic anchor (FACT): the wrong-subtitle-on-2nd-display defect was "fixed" via a source-side

`CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class recurred silently while the GREEN suite passed. Industry-standard bug-driven testing (Google content-driven testing; AOSP CTS/Tradefed) made mechanical + enforced. Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + recommended gates `CM-REGRESSION-GUARD-REGISTERED` / `CM-REGRESSION-GUARD-SUITE-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag.

§11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08). Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user's path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate: "too many dropped buffers" >25, "unexpected presentation timestamp", "test timed out"). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (BBC ExoPlayer testing samples) is the offline ground-truth. Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback`, `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, 2026-06-08). Refines §11.4.117 + §11.4.107 + §11.4.112. Forensic anchor (FACT): tests tasked to "physically verify the 2nd-display subtitle" PASSEd GREEN while subtitles did NOT show / showed WRONG — the streaming player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (autonomous PIXEL verification structurally impossible per §11.4.112), so the test fell back to the accessibility-scraped/`persist.atmosphere.subdebug proxy`, and the proxy accepted a chrome/menu LABEL (`Аудио и субтитры`) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed. The mandate: a subtitle-correctness test MUST classify the cue's *content class* — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/menu label (closed multilingual deny-list MIRRORED from source, case-folded incl. non-ASCII), time/numeric chrome, not prose, OUTSIDE the lower safe-title band (CEA-708 9-anchor grid), OR STATIC across the window (real subtitle changes → ≥ 2 distinct prose cues, a metamorphic relation). DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥ 2 OR fuzzy-matches the SOURCE-extracted cue via normalized edit distance (§11.4.123 host ground truth). The oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern forbidden here. Secure-display honesty (§11.4.112): where FLAG_SECURE makes pixel verification impossible, the rock-solid autonomous proof is the player's caption telemetry + source-track presence + content-class oracle — NEVER a faked pixel "physical" pass; human-eye pixel confirmation is `operator_attended` (§11.4.52) with a tracked migration item. App-agnostic (keys off content class). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-137-PROPAGATION` (literal `11.4.137`) + recommended gate `CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE` + paired §1.1 mutation (strip the denylist/position/cadence check → golden-bad `Аудио и субтитры` PASSes → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.137. Non-compliance is a release blocker. No escape hatch — no `--present-cue-is-correct` , `--skip-chrome-oracle` , `--length-heuristic-suffices` , `--pixel-pass-on-secure-display` , `--skip-position-check` , `--skip-cadence-check` flag.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, 2026-06-08). When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to `file:line` (canonical example: `lib/subtitle_content_validation.sh:sub_is_prose()` returning TRUE for `Аудио и субтитры`); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under `docs/research/<scope>/<defect>_bluff_audit/`. Closing an operator-found defect WITHOUT the bluff-audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-138-PROPAGATION` (literal `11.4.138`) + recommended gate `CM-OPERATOR-ESCAPE-BLUFF-AUDIT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.138. Non-compliance is a release blocker. No escape hatch — no `--close-without-bluff-audit`, `--operator-find-is-just-a-bug`, `--skip-permanent-guard` flag.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08). Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert `running-artifact == built-artifact`: the deploy yielded a CLEAN target (mutable-overlay/userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature

is "the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label." Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents **MUST** cut token spend (input AND output) toward **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance prefix** — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost $\sim 0.1\times$ base input (the dominant cost driver — measured $\sim 170\text{K}$ tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350–520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal `11.4.N` token so propagation gates pass) with the canonical full text kept gate-scanned in `constitution/Constitution.md` and reachable in one hop — a deduplication realising §11.4.35, never a deletion; (4) **CodeGraph/retrieval-first over full-file loading** (§11.4.78/§11.4.79); (5) **output-token reduction** — terse status + `effort:"low"` on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions**. **Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative

usage object (input/cache_read/cache_creation/output split; NEVER tiktoken , NEVER the client-side cost estimate), reproduced N times (§11.4.50), pass = AFTER ≤ 40% of BEFORE OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run MUST show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (cache_read_input_tokens > 0) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the *measured* reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes §11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-141-PROPAGATION (literal 11.4.141) + recommended gate CM-TOKEN-EFFICIENCY + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses → measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.141. **Project instantiation (§11.4.35):** [the consuming project fills in: confirm the Claude Code governance-prefix cache is warm + free of pre-breakpoint volatiles; tier the §11.4.96-SAFE mechanical subagents (search/grep/status/doc-export) to Haiku with output-to-file under qa-results/ ; replace the consumer CLAUDE.md 112-row Applied-Fixes inline table + verbatim anchor restatements with a literal-anchor index pointing at constitution/Constitution.md ; keep CodeGraph/lumen-first navigation; run the harness BEFORE/AFTER to prove the measured reduction.] Non-compliance is a release blocker. No escape hatch — no --skip-token-efficiency , --no-cache-governance , --assert-reduction-without-measuring , --tier-down-reasoning , --inline-all-governance , --tiktoken-estimate-OK flag.

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: "ALL changes we do MUST pass through the code review step!!! ALWAYS!!!" EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN),

conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125's "after all fixes/changes/ implementations are done" phrasing could leave open — no "just a doc edit", no "just a one-liner", no "the author already self-reviewed (§11.4.92)", no "trivial change" carve-out. **Independence is load-bearing** — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92's multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp "looks good" is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate `CM-COVENANT-114-142-PROPAGATION` (literal `11.4.142`) + recommended gate `CM-EVERY-CHANGE-REVIEWED` (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → `CM-EVERY-CHANGE-REVIEWED` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.142. Non-compliance is a release blocker. No escape hatch — no `--skip-review`, `--no-review`, `--trivial-change-exempt`, `--doc-edit-exempt`, `--self-review-suffices`, `--review-after-commit` flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: "All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that's it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + a consuming project's extensions." Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app's OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app's own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — "the show I picked actually plays" — unproven (the operator's "we just eventually play on 2nd display sample, and that's it mostly" gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app's own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target) via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title's captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app's accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is `operator_attended` SKIP-with-reason per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): "the routing fired so the title is playing" is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it.

Composes §11.4.48 / §11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate `CM-COVENANT-114-143-PROPAGATION` (literal `11.4.143`) + recommended gate `CM-VIDEO-REAL-JOURNEY-TEST` (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPS-with-reason) + paired §1.1 mutation (replace a real-journey test's browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.143. Non-compliance is a release blocker. No escape hatch — no `--sample-playback-ok`, `--skip-real-journey`, `--deep-link-suffices`, `--routing-only-pass`, `--no-title-selection` flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project's ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project's recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the

project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35.

Propagation gate `CM-COVENANT-114-144-PROPAGATION` (literal `11.4.144`) + recommended gate `CM-DEVICE-AVAILABILITY-FOLLOWED` + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.144. Non-compliance is a release blocker. No escape hatch — no `--skip-availability-following` , `--silent-recording-gap-OK` , `--no-reconnect-wait` , `--invent-reconnect-timing` , `--skip-recovery-escalation` , `--autonomous-power-cycle-OK` flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial "refute-the-change" stance to defeat the documented LLM confirmation-bias failure mode) MUST research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/

contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec AND the connected features' contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review yet failed at RUNTIME ONLY (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 SOURCE → ARTIFACT → RUNTIME → USER-VISIBLE gap caught only after the fact; §11.4.145 shifts that discovery LEFT. Each angle MUST name the TOOL(S) used + obtain the required DEPTH (the diff, the full changed unit, its declared contract, the spec section, every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/ §11.4.69 ("looks fine" forbidden, §11.4.1); a genuinely-N/A angle is recorded

NOT_APPLICABLE: <reason> per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/ §12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-145-PROPAGATION (literal 11.4.145) + recommended gate CM-IMPACT-RESEARCH-PER-CHANGE + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → CM-IMPACT-RESEARCH-PER-CHANGE FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule

Constitution.md §11.4.145. Non-compliance is a release blocker. No escape hatch — no --skip-impact-research, --single-angle-suffices, --author-researches-own-change, --diff-skim-OK, --no-go-overridable, --trivial-change-no-research flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator's exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped `RED_MODE=1→0`) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never "no other issues found"); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per §11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised

cases stated as honest gaps, never silently implied clean. Composes §11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/ §1.1. Classification: universal (§11.4.17). Propagation gate

CM-COVENANT-114-146-PROPAGATION (literal 11.4.146) + recommended gate CM-REPRODUCE-FIRST-THEN-EXTEND (every closed defect's fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its RED_MODE=0 GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce → confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce → confirm pair and no extend-case-set →

CM-REPRODUCE-FIRST-THEN-EXTEND FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.146. Non-compliance is a release blocker. No escape hatch — no

```
--skip-reproduce-first , --fix-without-red , --skip-extend-to-all-cases , --reproduce-confirm-suffices ,  
--defer-extend-to-release-sweep , --single-guard-suffices , --no-defect-characterisation flag.
```

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10).

Verbatim operator intent: "any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!" Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (API Error: Server is temporarily limiting requests rate-limit killed 5 at once, API Error: socket connection closed unexpectedly killed 2, one left PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — "it probably finished before dying" is a forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has

an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch timestamp, status from the CLOSED SET {dispatched | in-flight | crashed | respawned | complete}), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; "an entry with no dispatch-event cannot show complete "); the registry is the SINGLE SOURCE OF TRUTH for "is this work owed?" and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime's own retries are exhausted) flips the entry to crashed + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches complete ; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never blocks the loop for a human), transient causes use the project's ALREADY-DEFINED backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve** → **§11.4.84-check** → **resume-or-clean-restart** — the crashed agent's uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation/ // always pass / _mutated_* residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent git worktree isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) "crash ≠ done"** **COMPLETION criterion** — a unit is DONE only when an agent reaches complete with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is dispatched / in-flight / crashed / respawned -not-yet- complete , the zero-idle survey (§11.4.94) treats every non- complete entry as an OPEN item to reclaim, and a registry showing complete without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40

retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect → wait/backoff → re-attach/respawn → escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/ §9.2/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-147-PROPAGATION` (literal `11.4.147`) + recommended gate `CM-CRASHED-AGENT-RESPAWN-TRACKED` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a `crashed` entry as the loop's done-condition `complete` without landed evidence, OR drop a dispatched agent without a registry entry → `CM-CRASHED-AGENT-RESPAWN-TRACKED` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.147. Non-compliance is a release blocker. No escape hatch — no `--skip-agent-registry`, `--crash-equals-done`, `--no-respawn`, `--discard-partial-state`, `--blind-commit-partial`, `--forget-dead-agent`, `--loop-done-with-crashed-entries` flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSOT / §11.4.106 docs_chain) into ONE integrity contract spanning DB ↔ docs ↔ external tracker, adding the operator's three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB `description` column AND the docs AND the tracker, never a stub/§11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every `Operator-blocked` item (incl. `Blocked` / `BLOCKED` documented alias normalised to canonical `Operator-blocked`, never a silent fork §11.4.6) MUST enumerate the closed list of decisions/actions that would unblock it (`[A]...·[B]...·[C]...`, mirroring §11.4.66's 2–4-option shape); a `BLOCKED` item with no enumerated

choices FAILs; **(D4) regular never-missed bidirectional DB ↔ docs ↔ tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (db-to-md / md-to-db byte-identical round-trip §11.4.93 + tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key [`<ID>`] prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side `created=N updated=M failed=0` proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items + agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes §11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/ §1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-148-PROPAGATION` (literal `11.4.148`) + recommended gates `CM-ITEM-INTEGRITY-STATUS-TYPE-ID` / `CM-ITEM-COMPREHENSIVE-DESCRIPTION` / `CM-BLOCKED-UNBLOCK-CHOICES` / `CM-TRACKER-SYNC-IDEMPOTENT` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.148. Non-compliance is a release blocker. No escape hatch — no `--item-without-status`, `--item-without-type`, `--item-without-id`, `--stub-description-OK`, `--blocked-without-choices`, `--skip-tracker-sync`, `--one-way-sync-OK`, `--non-idempotent-tracker-push`, `--hardcode-assignee` flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 `item_history` (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles):

the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a `test_diary` table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC `date_time` + `tested_by` (closed set `User|Operator|AI-agent|HelixQA`) + `result` (§11.4.45 `PASS|FAIL|SKIP` + detail) + in-depth observations (long markdown, facts or explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` §11.4.6, the background a future fix needs) + `action_taken` + `status_changed` +from/to (did this run change status + WHY) + §11.4.69 `evidence_path` + `feature_class`, with a SCHEMA CONSTRAINT making a `PASS` row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item `Diary / Diary_Summary` exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 `docs_chain`-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK model — the item task's description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type `task`) with a `{TODO|In-progress|Completed}` lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; **(e)** MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the `observations` prose is authored by whoever ran the test; tooling only stores/renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run's verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/ §1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary doc layout, export formats, tracker sub-task field map, `docs_chain` context per §11.4.35. Propagation gate `CM-COVENANT-114-149-PROPAGATION` (literal §11.4.149) + recommended gates `CM-TEST-DIARY-SYNC` / `CM-DIARY-PASS-REQUIRES-EVIDENCE` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.149. Non-compliance is a release blocker. No escape hatch — no `--skip-testing-diary` , `--diary-without-evidence` , `--no-diary-summary` , `--diary-without-tracker-subtask` , `--llm-in-diary-data-path` , `--diary-export-optional` flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: "For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don't have a more serious problem we're unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!" For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation, API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted `won't-fix: structurally-impossible` (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a "we can't" became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/ structural WITHOUT a documented deep-research pass** — no item may be marked `Fixed` / `Implemented` / `Completed` (§11.4.33) OR classified `structurally-impossible won't-fix` (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes §11.4.8's pass UNCONDITIONAL (every item,

however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; "we found nothing worse" requires the enumerated search (§11.4.118). **(D) Latest-source + cited** — LATEST authoritative versions per §11.4.99 (never training-data/memory), each cited by URL + access date in the research artefact AND the closure commit footer (`Deep-research <date>: <urls>` OR the literal `NO external solution found – original work` per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed $\leftrightarrow$ Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and "we probably don't have a bigger problem" without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate `CM-COVENANT-114-150-PROPAGATION` (literal `11.4.150`) + recommended gate `CM-DEEP-RESEARCH-PER-ISSUE` (every closed/structural-verdicted item carries a documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation

(strip the literal → propagation gate FAILs; close an item or reach a structural verdict with no documented multi-angle research artefact / cited footer →

CM-DEEP-RESEARCH-PER-ISSUE FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.150. Non-compliance is a release blocker. No escape hatch — no `--skip-deep-research`, `--single-source-suffices`, `--trivial-change-no-research`, `--close-without-research`, `--structural-without-research`, `--serialise-research`, `--research-from-memory-OK` flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: "Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project's release prefix, e.g. `myproject-1.0.0-dev-0.0.1`. The prefix MUST come from `HELIX_RELEASE_PREFIX` in our `.env` if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository." Every release tag AND every version name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project's release prefix, form `<PREFIX>-<version>` (canonical example: `myproject-1.0.0-dev-0.0.1`), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1)

`HELIX_RELEASE_PREFIX` from the project's `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked `.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo `<PREFIX>-1.2.0` while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (`<PREFIX>-...-0.0.1` → `<PREFIX>-...-0.0.2` → ...), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a

release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var per §11.4.35. Propagation gate `CM-COVENANT-114-151-PROPAGATION` (literal `11.4.151`) + recommended gate `CM-RELEASE-PREFIX-NAMING` (every release tag/version on the main repo + every owned submodule carries the resolved `<PREFIX>-` prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → `CM-RELEASE-PREFIX-NAMING` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.151. Non-compliance is a release blocker. No escape hatch — no `--no-release-prefix`, `--unprefixed-tag`, `--prefix-optional`, `--differing-submodule-prefix` flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13).

Verbatim operator intent: "For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!" Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green). STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory,

neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL four surfaces** — fatal crashes, ANRs, performance traces/regressions, AND non-fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), "acknowledged in console" is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: `RED_MODE=1` captures the recorded defect on the pre-fix artifact, `RED_MODE=0` standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4.1/.6 — "no new issues" requires the enumerated monitored-surfaces+window §11.4.118; "fixed" requires the RED → GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is "closed" only when its guard is GREEN on a clean artifact, never when the console mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is a consuming project's §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a

non-fatal with triage context). Composes

§11.4.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/

§1.1. Propagation gate `CM-COVENANT-114-152-PROPAGATION` (literal `11.4.152`)

+ recommended gate `CM-CRASHLYTICS-ISSUE-FULLY-COVERED` (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md`

§11.4.152. Non-compliance is a release blocker. No escape hatch — no `--skip-crashlytics-monitoring` , `--monitor-crashes-only` , `--skip-non-fatals` , `--resolve-without-regression-test` , `--console-mark-is-fixed` , `--monitor-once` , `--mute-without-rationale` flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15).

Every project MUST maintain, under `docs/features/` , a comprehensive feature Status document set (`Status.md` + its §11.4.56 `Status_Summary.md` companion) enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component → Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer §11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible "confirmed" claim backed by a recorded real-use video of a genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mockd/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35);

a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146 retest → re-record → clean GO per §11.4.134 before "confirmed" (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60. (8) **MP4 format REQUIRED**. All video confirmations MUST be in .mp4 format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). .cast files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row's §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146/ §1.1. Propagation gate CM-COVENANT-114-153-PROPAGATION (literal 11.4.153) + recommended gates CM-FEATURE-STATUS-COMPLETE + CM-FEATURE-STATUS-VIDEO-CONFIRMED + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.153. Non-compliance is a release blocker. No escape hatch — no --skip-feature-status , --feature-without-video , --frozen-video-OK , --bluff-response-video-OK , --skip-video-analysis , --feature-ledger-incomplete-OK , --no-docx-export , --allow-feature-ledger-drift flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: "recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!" + "All old recording files MUST BE removed when new one starts!" Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI

terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): "remove old" = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed docs/qa/<run-id>/ evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + recommended gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.154. Non-compliance is a release blocker. No escape hatch — no --whole-screen-capture-OK , --skip-window-scope , --keep-stale-recordings , --no-corpus-rotation , --full-desktop-recording flag. **(C) MP4 auto-conversion REQUIRED.** Any .cast file produced MUST be auto-converted to .mp4 via agg + ffmpeg immediately after capture. The .mp4 is the primary evidence; .cast is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: "All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution." Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed docs/qa/<run-id>/ trail (§11.4.83) —

MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) `HELIX_RELEASE_PREFIX` from `.env` (authoritative, git-ignored §11.4.30, documented in tracked `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so `ls '<PREFIX>--- '*` enumerates the corpus; canonical form `<PREFIX>---<feature-or-scope>---<run-id>.<ext>` (`---` delimits the prefix unambiguously); MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation (rotation removes the agent's OWN `<PREFIX>--- *` only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1. Propagation gate `CM-COVENANT-114-155-PROPAGATION` (literal `11.4.155`) + recommended gate `CM-RECORDING-PROJECT-NAME-PREFIX` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix`, `--recording-without-project-name`, `--unprefixed-recording`, `--prefix-optional-for-recording`, `--differing-recording-prefix` flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: "Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!" Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline,

or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any on: push / schedule / workflow_dispatch). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a ...disabled-local-only non- .yaml name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active .github/workflows/*.yaml|yaml / .gitlab-ci.yaml / .gitlab/** / equivalent at the ROOT of any governed repo/submodule (the only place a provider executes); **(B)** "disabled" = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 .disabled / .disabled-local-only); a live- on: + if:false workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs below the root (AOSP external/** , prebuilts/** , vendored submodules) are INERT (a provider never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = "does a push to OUR upstream trigger a run?" yes⇒disable, inert⇒document+leave (§11.4.6 verify-not-assume); **(D)** no new CI may be added (release blocker); **(E)** pre-push verify git ls-files | grep -E '^\.github/workflows/.*\.ya?ml\$|^\.gitlab-ci\.yaml\$' empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection required checks, provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-156-PROPAGATION (literal 11.4.156) + recommended gate CM-NO-ACTIVE-CI + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root .github/workflows/x.yaml to an authored repo → CM-NO-ACTIVE-CI FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.156. Non-compliance is a release blocker. No escape hatch — no --allow-ci , --enable-workflow , --keep-pipeline , --remote-ci-OK , --ci-exempt flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator mandate: "Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain

GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSTITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!" Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4 propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal `11.4.N` anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md "in sync" while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-157-PROPAGATION` (literal `11.4.157` , GEMINI.md INCLUDED) + recommended gate `CM-GEMINI-MD-LOCKSTEP` + paired §1.1 meta-test mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.157. Non-compliance is a release blocker. No escape hatch — no `--skip-gemini-md` , `--gemini-optional` , `--gemini-lag-OK` , `--four-carrier-suffices` flag.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts → real responses → real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→ §11.4.153(4)/§11.4.4 fix → retest → re-record), never a confirmation. The testing System MUST ACTUALLY READ every

shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — "a video was produced" is NOT evidence, "the System read the screen + confirmed a genuine result" is. HelixQA (§11.4.27) MUST drive this exercise → record → read → score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = `$HOME/Downloads` (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. **Vision analysis MANDATORY for every recording** — after every recording, the agent MUST read the terminal output / video content and verify the feature actually works. A recording without a vision-confirmed verdict is a §11.4.158 violation. Composes §11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/ §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-158-PROPAGATION` (literal `11.4.158`) + recommended gates `CM-INTENSIVE-RECORDING-COVERAGE` + `CM-RECORDING-CONTENT-READ-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.158. Non-compliance is a release blocker. No escape hatch — no `--skip-recording-coverage`, `--video-without-content-read`, `--happy-path-recording-suffices`, `--recording-path-anywhere`, `--unread-recording-OK`, `--skip-helixqa-read` flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen; use macOS `screencapture -l<window_id>`, Linux `xdotool + ffmpeg`, or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** —

`.mp4` (H.264, `movflags +faststart` , `pix_fmt yuv420p`); `.cast` files are supplementary only; auto-convert via `agg + ffmpeg` per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from `HELIX_RELEASE_PREFIX` in `.env` (§11.4.151) or lowercased project root dir name (§11.4.29); format `<project_name>-<feature>-<timestamp>.mp4` ; unprefixed = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no "TODO implement" / "simulate" / "for now" patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via `osascript` / `xdotool`); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix → retest → re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent's own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY — not duration-based** — the value of a recording is NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just "something ran"), LLM responses are REAL content (not simulated, not placeholder, not empty), every claim of "working" is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected**

recordings — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent **MUST** investigate **WHY** before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding **WHY** the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output **DURING** recording (not after); this catches issues immediately and allows corrective action before the recording completes. Classification: universal (§11.4.17). Composes §11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1. Propagation gate `CM-COVENANT-114-159-PROPAGATION` (literal `11.4.159`) + recommended gate `CM-WINDOW-VIDEO-VALIDATED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.159. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-ok`, `--cast-only`, `--skip-vision-validation`, `--no-cleanup`, `--simulated-recording-ok`, `--unprefixed-recording` flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence **MUST** be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results **BEFORE** acceptance; the recording system **MUST** provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); the bridge **MUST** capture frames at $\leq 5s$ intervals, run OCR/vision analysis with a self-validated golden-good/golden-bad analyzer (§11.4.107(10)), compare extracted text against specified patterns, produce a per-frame PASS/FAIL verdict with an evidence path to the frame, and surface failures immediately so the recording can be re-done per §11.4.159(L). The capture interval, OCR confidence floor, and pattern-matching thresholds are project-configured per §11.4.35, calibrated on the project's own fixtures (§11.4.6). Honest boundary (§11.4.6): vision verification confirms the feature produced expected on-screen output — it does **NOT** replace §11.4.5 captured-evidence quality analysis (audio/video metrics) nor §11.4.108 runtime-signature verification; FLAG_SECURE/DRM-protected/sink-blanked surfaces document the gap per §11.4.112 and use the §11.4.117 proxy oracle instead. Classification: universal (§11.4.17). Composes

§11.4.5/.27/.69/.107(10)/.112/.117/.153/.158/.159(J)/§1.1. Propagation gate CM-COVENANT-114-160-PROPAGATION + recommended gate CM-VISION-VERIFIED-RECORDING-BRIDGE + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21).

Compact summary: every project MUST use Podman in rootless mode (or equivalent rootless container runtime) for ALL containerized workloads — Docker in rootful mode, sudo, or any escalation to root for container management is STRICTLY FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the vasic-digital/containers Submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad-hoc docker/podman commands outside the Submodule's pkg/boot / pkg/compose / pkg/health primitives; if a missing capability forces a raw command, the containers Submodule MUST be extended upstream per §11.4.74 rather than worked around; all container-related integration tests MUST boot infra on-demand via the Submodule (the on-demand-infra invariant). Honest boundary (§11.4.6): rootless execution eliminates the container-to-root privilege-escalation vector — it does NOT replace §11.4.10 credentials-handling (credentials in containers still require leak audits) nor §12.3 container hygiene (memory limits/OOM policies/restart backoff still apply). Classification: universal (§11.4.17). Composes §11.4.76/.74/.10/.112/§12.3/.6. Propagation gate CM-COVENANT-114-161-PROPAGATION + recommended gate CM-ROOTLESS-CONTAINER-RUNTIME + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project producing user-facing interfaces (web, desktop, mobile, TUI) MUST use OpenDesign (<https://github.com/nexu-io/open-design>) as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes system for: (a) all color palette definitions supporting BOTH light and dark themes from project brand assets (§11.4.35), (b) typography scale and spacing, (c) component-level design tokens; if a desired UI pattern is not supported, extend OpenDesign upstream per §11.4.74 (extend-don't-reimplement); every UI

component MUST ship light + dark theme variants; elements MUST NOT overlap, fonts MUST NOT collide, labels MUST NOT overlay labels — any layout regression is a §11.4.162 violation; all UI changes MUST be covered by the project's standard test types including visual regression tests (before/after screenshots with per-pixel or perceptual-diff PASS/FAIL). Honest boundary (§11.4.6): OpenDesign governs design tokens and themes — it does NOT replace functional testing (§11.4.27), WCAG accessibility assertions (§11.4.107), nor the §11.4.48/§11.4.49 UI-driven and dual-approach testing methodology. Classification: universal (§11.4.17). Composes §11.4.74/.25/.27/.4(b)/.107/.48/.49/.35/.69/§1.1. Propagation gate CM-COVENANT-114-162-PROPAGATION + recommended gate CM-OPENDESIGN-UI-SYSTEM + paired §1.1 mutation.

§11.4.163 — Universal Media Validation & Verification Mandate (User mandate, 2026-06-21). Compact summary: every recorded artifact (video MP4, audio WAV, screenshots PNG, asciinema cast, text output) MUST pass through a MEDIA VALIDATION pipeline before acceptance — OCR for video/screenshots with confidence floor (§11.4.117/.107(12)), transcription for audio, text parsing for asciicast; compare extracted content against SPECIFY-phase expected patterns (§11.4.159(J)); self-validated analyzer with golden-good/golden-bad fixture pair (§11.4.107(10)) — golden-bad MUST FAIL or the pipeline is a bluff gate; produce structured verdict (PASS/FAIL + evidence path + matched/unmatched patterns + pinpoint data on FAIL — which frame/line/timestamp, expected vs actual); triggerable as post-recording AND real-time step (§11.4.159(M)); paired §1.1 mutation ensures golden-bad fixture produces FAIL. Honest boundary: confirms artifact content matches intended patterns — does NOT replace §11.4.5 quality analysis, §11.4.107 liveness, nor §11.4.108 runtime-signature. Classification: universal (§11.4.17). Composes §11.4.107(10)/.112/.117/.159(J)/.159(K)/.159(M)/.83/.5/.69/.102/.134/§1.1. Propagation gate CM-COVENANT-114-163-PROPAGATION + recommended gate CM-MEDIA-VALIDATION-PIPELINE + paired §1.1 mutation.

§11.4.164 — Universal Constitution Auto-Propagation & Hook System (User mandate, 2026-06-21). Compact summary: every fetch+pull of the constitution submodule MUST trigger `constitution/scripts/post_update_hook.sh` (inherited by reference §11.4.28, NEVER copied) that: (a) detects which files changed (Constitution.md/CLAUDE/AGENTS/QWEN/GEMINI + scripts/ hooks/ skills/ mcp/ plugins/) via `git diff --name-only` ; (b) registers new/modified skills with the agent's skill system via consumer's `scripts/register_skills.sh` ; (c) registers new/modified MCP servers via consumer's `scripts/register_mcp.sh` ; (d) installs new/modified hooks into `.git/hooks/` ; (e) makes scripts executable + validates syntax (`sh -n / bash -n` per §11.4.67); (f) emits a summary to stdout + `.constitution/state/last_update.log` ; (g) on failure logs exact file:line and continues (§11.4.6); (h) tested by §1.1 mutation (add skill → hook detects it). Consumer MUST invoke hook after every constitutional pull; failure to invoke is non-compliance. Honest boundary: installs components — does NOT guarantee consumer runtime acceptance nor replace §11.4.32 validation sweep. Classification: universal (§11.4.17). Composes §11.4.28/.32/.35/.67/.77/.102(B)/.122/.31/§1.1. Propagation gate `CM-COVENANT-114-164-PROPAGATION` + recommended gate `CM-CONSTITUTION-AUTO-PROPAGATION` + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent Mandate (User mandate, 2026-06-21). Compact summary: every code change OR recorded media artifact MUST pass an INDEPENDENT verifier structurally separate from the author (§11.4.70/.20, NEVER author self-review — §11.4.92 PRECEDES, never satisfies), producing structured findings with evidence paths and iterating to GO per §11.4.134. (a) CODE: §11.4.142 review + build+test diff scope + run paired §1.1 mutations + verify §11.4.108 runtime-signature registry; (b) MEDIA: run §11.4.163 pipeline + confirm genuine content (no mock/stub/placeholder §11.4.159(I)) + confirm format (MP4 H.264/WAV/PNG/cast+mp4 §11.4.159(B)) + cross-check evidence path; (c) DOCS: validate HTML+PDF+DOCX exports current §11.4.65 + revision header incremented §11.4.44 + §11.4.86 fingerprint match; (d) CONFIG: validate YAML/JSON/TOML syntax + schema cross-reference + §11.4.10 leak check; (e–f) structured findings, iterate to zero-finding GO; (g) verifier self-validated by §1.1 mutation (knowingly broken change MUST be caught). Honest boundary: confirms source/artifact integrity — does NOT replace §11.4.108 runtime-signature

nor §11.4.40 full-suite retest. Classification: universal (§11.4.17). Composes §11.4.142/.163/.44/.65/.86/.108/.125/.134/.10/.40/.92/.70/.20/.102/.159(I)/.159(B)/ §1.1. Propagation gate `CM-COVENANT-114-165-PROPAGATION` + recommended gate `CM-INDEPENDENT-VERIFICATION-AGENT` + paired §1.1 mutation.

§11.4.166 — REPEALED (operator decision, 2026-06-22). The Universal Semgrep static-analysis mandate is repealed — Semgrep is NO LONGER mandatory. Static analysis remains encouraged, not mandated. Scaffolding, `submodules/semgrep` , MCP/pre-commit/PATH wiring, `docs_chain semgrep_status` context, and the `CM-COVENANT-114-166-PROPAGATION` / `CM-SEMGREP-WIRED` gates removed. Anchor 11.4.166 retired. See constitution `Constitution.md` §11.4.166.

§11.4.167 — Big-work-item feature work-stream lifecycle (User mandate, 2026-06-23). Compact summary: every BIG work item (new feature OR drastic/ large fix) MUST be developed as its own isolated **feature work-stream** — its OWN full project copy in a sibling directory, its OWN `feature/<slug>` branch, its OWN per-feature flashable builds + feature release tags, kept SEPARATE from the trunk until the operator manually approves after testing, with the trunk regularly merged INTO every feature stream. (A) one big item = one feature work-stream in a sibling project copy (`<project>_<slug>` , §11.4.29 slug); small changes stay on the trunk. (B) disk-feasible duplication MANDATORY, naive deep copies FORBIDDEN — a CoW/reflink clone (`cp -a --reflink=auto` / FS snapshot) on btrfs/XFS/ZFS/ APFS, else a shared-object-store mechanism (`git worktree` + externalized build output); per-stream `out/` excluded from the clone + one shared ccache; ~0-disk-per-fresh-stream verified by captured evidence (§11.4.69) before relied on (§11.4.6 no-guessing). (C) own branch + own tags greppable per §11.4.151 (`<base>-feat-<slug>[-<iter>]`), NEVER merged to trunk until §11.4.40 full retest GREEN on a clean target → §11.4.41 merge-first → §11.4.113 ff-only (no-force) merge. (D) trunk merged INTO every live stream FREQUENTLY (per trunk tag / daily, `git merge` never rebase a shared/tagged branch); a stale stream is flagged. (E) feature branch + mirrored feature tag cascades to EVERY touched submodule the moment a submodule is modified (§11.4.113 + tag-cascade); untouched submodules stay

trunk-pinned. (F) single-builder build QUEUE (FIFO + advisory lock, §11.4.58 L1 / §12.7 / §12.8) + finite-device test queue (§11.4.119 per-device exclusive ownership); idle streams' regenerable build output GC'd (§11.4.77). (G) every change crosses the full quality gauntlet (§11.4.142/.125/.134 review + §11.4.145 impact research + §11.4 / .5 / .69 / .107 anti-bluff testing) — a feature stream is NOT a quality carve-out. (H) resumable, data-safe lifecycle — a stream registry (§11.4.116 / §11.4.147) tracks branch/base-tag/out-state/last-trunk-merge/merge-approval; atomic `.partial` → rename create + §11.4.84 quiescence resume; retire only after merged/approved + §9.2 backup; bounded by §12 / §12.6. Honest boundary (§11.4.6): guarantees ISOLATION + disk-feasibility + greppable identity + controlled trunk ↔ feature merge — NOT that any stream's work is correct (still crosses §11.4.108 + §11.4.40). Classification: universal (§11.4.17) — consumer supplies its CoW mechanism, sibling-dir + tag naming, build/device-queue counts, big-item threshold per §11.4.35. Propagation gate `CM-COVENANT-114-167-PROPAGATION` (literal `11.4.167`) + recommended gates `CM-FEATURE-WORKSTREAM-COW-CLONE` / `CM-FEATURE-WORKSTREAM-NO-MERGE-UNTIL-APPROVED` / `CM-FEATURE-WORKSTREAM-TRUNK-SYNC-CADENCE` / `CM-FEATURE-WORKSTREAM-SUBMODULE-CASCADE` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.167. Non-compliance is a release blocker. No escape hatch — no `--full-clone-per-feature`, `--deep-copy-stream`, `--skip-cow-clone`, `--merge-feature-without-approval`, `--no-trunk-sync`, `--skip-submodule-cascade`, `--unbounded-parallel-builds`, `--shared-device-no-lock`, `--retire-unmerged-stream`, `--feature-stream-quality-carveout` flag.

§11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate, 2026-06-23). Compact summary: every generated/exported document (HTML / PDF / DOCX / any format in the §11.4.65 export scope) MUST pass INDEPENDENT validation by a review agent structurally separate from the generator (§11.4.70/§11.4.142, NEVER the author self-checking), on BOTH the source AND every exported artifact, ALWAYS, across THREE layers: (1) CONTENT — the export faithfully carries the source's intent + data, nothing dropped/truncated/garbled; (2) TEXTUAL — human-readable, NO raw markup / diagram-source (Mermaid `gantt` / `graph` / `flowchart` / `sequenceDiagram` etc.) / unrendered code-fence leaking as body text (the exact 2026-06-23 raw-gantt-in-

PDF failure); (3) FULL VISUAL — embedded diagrams render as IMAGES not source, layout intact, no overlap/cut-off/garble/collapse, verified by RENDERING the export (`pdftotext` catches raw source as text + `pdfimages` confirms rendered images + `pdftoppm` → OCR confirms human-readable visual content) with captured evidence per §11.4.5/§11.4.69/§11.4.107. Anti-bluff: a rubber-stamp is not validation; the analyzer is self-validated golden-good/golden-bad per §11.4.107(10) (a golden-bad export seeded with raw `gantt` source MUST FAIL); iterate to a clean GO per §11.4.134. Forensic FACT (2026-06-23): a "green" Mermaid fix shipped raw `gantt` source as readable text into user-facing PDFs because validation grepped the HTML for `class="mermaid"` and NEVER checked the exported PDF — operator caught it (§11.4.138). A generated doc shipping without this content+textual+visual validation, or containing raw diagram source / unreadable garble, is a §11.4 PASS-bluff at the documentation layer. Honest boundary (§11.4.6): confirms the exported artifact a reader opens is faithful/readable/visually-rendered — does NOT prove the source content is correct (§11.4.142/§11.4.135) nor replace the §11.4.65 mtime-parity freshness gate (this is the READABILITY/FIDELITY layer the mtime gate cannot see); FLAG_SECURE/un-rasterisable surfaces document the gap per §11.4.112 + §11.4.117 proxy oracle, never a faked visual PASS. Classification: universal (§11.4.17). Composes §11.4.65/.73/.107/.117/.135/.138/.142/.159/.163/.165/§1.1. Propagation gate `CM-COVENANT-114-168-PROPAGATION` (literal `11.4.168`) + recommended gate `CM-EXPORTED-DOC-VISUALLY-VALIDATED` + paired §1.1 mutation (a PDF/HTML/DOCX containing raw diagram source as body text → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.168. Non-compliance is a release blocker. No escape hatch — no `--skip-visual-validation` , `--raw-source-in-pdf-ok` , `--textual-check-suffices` , `--self-validate-export` , `--diagram-source-as-text-ok` flag.

§11.4.169 — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, 2026-06-25). Every project MUST have, with coverage aiming 100% and zero false results / rock-solid hard physical proof / zero bluff, this CLOSED enumerated test-type set: (1) **unit** (the ONLY layer mocks/stubs/fakes/placeholders are permitted §11.4.27(A)); (2) **integration** (real fully-wired components against real infra, no fakes beyond unit, infra booted on-demand via the **containers** submodule §11.4.76/§11.4.161); (3) **e2e** (full user journey

across the real System §11.4.5/§11.4.107); (4) **full-automation** (fully autonomous, re-runnable, NO manual step after start §11.4.25/§11.4.52/§11.4.98, deterministic across N iterations §11.4.50); (5) **Challenges** (the **challenges** submodule `vasic-digital/challenges` — anti-bluff Challenge banks scoring PASS only on positive captured evidence §11.4.27(B)/§11.4.4(b) layer 4); (6) **HelixQA** (the **helix_qa** submodule with proper written **test banks (suites)** per application/service/platform AND comprehensive **fully-autonomous QA sessions** driving every registered bank with captured wire evidence per check §11.4.27); (7) **DDoS/load-flood** (sustained adversarial traffic — refuse cleanly OR degrade gracefully, never collapse; captured throughput/latency/error evidence); (8) **security** (authn/authz, injection/taint, secret-leak §11.4.10, transport+crypto, dependency CVEs, default-deny; composes the **security** submodule); (9) **stress+chaos** (sustained load + concurrent contention + boundary inputs + failure-injection [process-death/network-fault/input-corruption/resource-exhaustion/state-corruption] with categorised recovery + captured evidence §11.4.85); (10) **concurrency/atomicity** (correctness under concurrent callers, no lost updates, transactional atomicity proven, idempotency under replay); (11) **race-condition/deadlock** (race detector + lock-order/deadlock analysis under contention; no blocking op inside a shared-lock region per §1; captured detector output is the evidence); (12) **memory** (leak census over soak, peak-RSS ceilings e.g. the iOS-NEPacketTunnelProvider-class budget, allocation/fragmentation profile; no unbounded growth across a 24h/N-iteration soak); (13) **benchmarking/performance** (p50/p95/p99 latency + throughput + resource cost vs a recorded baseline; a regression vs baseline is a finding). Each type is REQUIRED where the domain warrants it and each PASS cites rock-solid captured PHYSICAL evidence (§11.4.5/§11.4.69/§11.4.107); the ONLY permitted absence is an honest §11.4.3 SKIP-with-reason (topology/hardware/credential genuinely absent), NEVER a silent gap; a green result with no falsifiable captured evidence is a §11.4 PASS-bluff. Four-layer §11.4.4(b) enforcement of the suite (pre-build per-type-present+-executable + post-build + runtime + paired §1.1 mutation per type) + per-project coverage ledger (§11.4.25/§11.4.52) classifying every feature × test-type × evidence-state; a missing required type or an `OPERATOR_ATTENDED_ONLY` row is a release blocker until promoted or honestly SKIP-justified. §11.4.169 is the strict explicitly-enumerated expansion of §11.4.27. Composes/strengthens §11.4.4/.5/.6/.25/.27/.50/.52/.69/.76/.85/.98/.107/.123/.135/.146/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-169-PROPAGATION` (literal `11.4.169`) + recommended gate `CM-MANDATORY-TEST-TYPES-COVERED` +

paired §1.1 mutation (strip a required type or drop its evidence citation → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md`

§11.4.169. Non-compliance is a release blocker. No escape hatch — no `--skip-test-type`, `--partial-coverage-ok`, `--evidence-optional`, `--challenges-not-applicable`, `--skip-helixqa`, `--no-chaos`, `--no-ddos`, `--skip-memory-test`, `--skip-race-detector`, `--bench-optional` flag.

§11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate, 2026-06-25). Compact summary: every change to ANY user-facing UI surface (screen/component/view/widget/layout/theme on any platform — Android-Compose, iOS-SwiftUI/UIKit, web, desktop, TUI) MUST be proven by DEVICE-INDEPENDENT host-side RENDERED PIXELS before it may be claimed correct — the real component rendered to a PNG ON THE HOST (NO device, NO emulator, NO running app) via a host-render harness (Compose → Roborazzi/Paparazzi on the JVM; web → Playwright/Storybook snapshot; SwiftUI → snapshot-testing; equivalent per stack), for EVERY relevant screen×state×{light,dark} theme, with the PNG validated by BOTH (i) golden image-diff (per-pixel/perceptual PASS/FAIL) AND (ii) an OCR/vision oracle reading rendered text+labels+control bounds to assert legibility + NO overlap / NO label-over-label / NO clipping / NO off-screen control / NO collapsed-or-giant-unbounded widget (the §11.4.162 layout invariants PROVEN on real pixels). Forensic FACT (2026-06-25): UI work was "validated" by VALUE-EQUALITY unit tests (a hex colour string / an sp/dp size integer / a design-token field == a constant) that NEVER RENDERED A PIXEL — GREEN while the operator opened a broken unbounded giant-button screen. A value/token-equality / property-assertion UI test is FORBIDDEN as the PROOF a UI is correct (it may SUPPLEMENT, NEVER SUBSTITUTE the rendered-pixel proof); a "green" UI suite with no rendered-pixel artefact for the changed surface is a §11.4 PASS-bluff at the visual layer. Self-validated golden-good/golden-bad analyzer §11.4.107(10), thresholds calibrated on the project's own fixtures §11.4.6. "device offline" is NEVER a valid skip — host-render IS the device-independent path; honest §11.4.3 SKIP only where the platform genuinely has no host-render harness (tracked migration item, never a value-only fallback). Honest boundary §11.4.6: host-render proves the COMPONENT renders correctly device-independently per commit — it does NOT prove the running app on real hardware (that remains §11.4.153/.158/.159/.160's live-device recording + read-the-screen layer, which

§11.4.170 COMPLEMENTS not replaces — both mandatory), does NOT replace §11.4.162 OpenDesign token/theme governance (§11.4.170 PROVES the rendered RESULT of those tokens), DISTINCT from §11.4.168 exported-document visual validation. Classification: universal (§11.4.17). Composes §11.4.3/.5/.27/.40/.69/.107/.108/.117/.153/.158/.159/.160/.162/.163/.168/.169/§1.1. Propagation gate `CM-COVENANT-114-170-PROPAGATION` (literal `11.4.170`) + recommended gate `CM-HOST-RENDERED-UI-VISUAL-PROOF` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md`

§11.4.170. Non-compliance is a release blocker. No escape hatch — no `--value-assertion-proves-ui`, `--skip-rendered-pixel-proof`, `--token-equality-suffices`, `--device-offline-skip-visual`, `--single-theme-only`, `--unvalidated-image-diff-OK`, `--no-ocr-layout-oracle` flag.

§11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29). Every workable item (ATM-NNN ticket) in the project's workable-items database, `Issues.md`, `Fixed.md`, `Issues_Summary.md`, `Fixed_Summary.md`, and ALL derived documents MUST carry a comprehensive human-readable description that explains the item in plain, non-technical language suitable for non-developers (project managers, stakeholders, team leads, business partners). The description MUST be 5-7 sentences minimum, written so that ANY person — regardless of technical background — can understand: (a) WHAT the item is (the feature, fix, or task in simple terms); (b) WHY it matters (the user/business value); (c) HOW it works (the mechanism, without code references); (d) WHO benefits (the end user or stakeholder); (e) WHAT the expected outcome is (the measurable result). The description is MANDATORY — an item without a human-readable description is a §11.4 violation at the documentation layer. Descriptions MUST NOT use jargon, acronyms without expansion, code references, or technical implementation details as the primary explanation. Technical details MAY appear as supplementary context AFTER the plain-language explanation. The `description` column in the SQLite database (`docs/workable_items.db`) is the SINGLE SOURCE OF TRUTH for item descriptions; all derived documents (`Issues.md`, `Fixed.md`, summaries, exports) MUST carry the SAME description text. Pre-build gate `CM-HUMAN-READABLE-DESCRIPTION` (when implemented) walks every item in the DB and asserts the `description` field exists, is non-empty, and is ≥ 50 characters.

Paired §1.1 meta-test mutation strips a description → gate FAILs. Classification: universal (§11.4.17). Composes §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (column alignment), §11.4.91 (summary clarity), §11.4.93 (SQLite SSoT), §11.4.148 (item integrity), §11.4.65 (universal export). Propagation gate `CM-COVENANT-114-171-PROPAGATION` (literal `11.4.171`) + paired §1.1 mutation.

§11.4.172 — Mandatory production-readiness planning with realistic timeline projection (User mandate, 2026-06-29). Every project under this Constitution MUST maintain a living production-readiness planning document that: (a) provides REALISTIC timeline projections for all product milestones (MVP, beta, production-ready) based on actual development velocity data (items completed per week, reopening rate, blocking dependencies); (b) identifies ALL danger zones, weaknesses, and risk areas with mitigation strategies; (c) accounts for HW delivery timelines, licensing delays, and external dependency risks; (d) defines the critical path and identifies which items can slip without affecting MVP; (e) includes workstation/build-capacity analysis and optimization recommendations; (f) is updated at least monthly or whenever the item count changes by $\geq 10\%$; (g) is cross-referenced against the workable-items database for accuracy. The planning document MUST live at `docs/research/production_planning_<YYYYMMDD>/ANALYSIS.md` with HTML+PDF exports per §11.4.65. Honest boundary (§11.4.6): projections are estimates based on measured velocity — they guarantee the methodology is sound, NOT that specific dates will be met. Classification: universal (§11.4.17). Composes §11.4.6 (no-guessing — projections based on data, not wishful thinking), §11.4.40 (full retest before tag), §11.4.42 (iteration discipline), §11.4.93 (SQLite SSoT), §11.4.108 (four-layer verification). Propagation gate `CM-COVENANT-114-172-PROPAGATION` (literal `11.4.172`) + paired §1.1 mutation.

§11.4.173 — Containerized + distributed build mandate (User mandate, 2026-06-29). EVERY build of EVERY component (source compile, artifact/package/installer/container-image production, codegen, asset render — for ANY language/platform: Go, Android/Gradle, desktop, web, native, firmware) MUST run INSIDE a specialized build container provisioned via the `digital.vasic.containers`

submodule (§11.4.76) — NEVER on the bare host. The build containers MUST be DISTRIBUTED to the designated remote build host(s) (e.g. `thinker.local`) via the SAME containers-submodule distribution mechanism the infra uses (§11.4.76 Distributor / remote compose over SSH, §11.4.161 rootless), so the build EXECUTES on the remote build host (offloading the developer/main host); once the build completes the produced artifacts MUST be brought BACK to the originating main host (scp/rsync/volume copy) for use/flashing/distribution. Building outside a container, or on the bare host, is FORBIDDEN — a release blocker (the "works on my machine" / unreproducible-build class §11.4.76 exists to prevent). The build-host target + build-container definitions are config-injected (§11.4.28, never hardcoded in the submodule); a missing build-container capability is added by EXTENDING the containers submodule upstream (§11.4.74), never by an ad-hoc host build. Honest boundary (§11.4.6): the containerized+distributed build guarantees reproducibility + host-isolation + capacity offload — it does NOT replace the §11.4.40 full-suite retest, §11.4.108 four-layer artifact → runtime verification, or §11.4.38 installable-asset evidence (those still run against the brought-back artifact). Classification: universal (§11.4.17). Composes §11.4.76 (containers submodule — sole orchestration layer), §11.4.161 (rootless runtime), §11.4.74 (extend-don't-reimplement), §11.4.28 (config injection / decoupling), §11.4.24 (build-resource stats), §11.4.82 (iteration speedup — persistent caches in the container), §11.4.121 (no-commit-while-build-writes-artifacts), §11.4.38 (installable-asset evidence on the brought-back artifact), §11.4.108 (artifact → runtime verification), §12.6 (host memory ceiling — offloading the build preserves the main host). Propagation gate `CM-COVENANT-114-173-PROPAGATION` (literal `11.4.173`) + recommended gate `CM-CONTAINERIZED-DISTRIBUTED-BUILD` (every build runs via the containers submodule on the remote build host; a bare-host build is detected + FAILs) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.173. Non-compliance is a release blocker. No escape hatch — no `--build-on-host`, `--skip-container-build`, `--local-build-ok`, `--no-distributed-build`, `--bare-host-build` flag.

§11.4.174 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29). On any shared host where other projects/users/agents may run concurrently, every inspection of or action on a process, build, daemon, port, lock, or system-state signal MUST first positively VERIFY the target is OURS

before diagnosing or acting. Attribute a process to us ONLY by a strong discriminator — cwd inside this project's tree, argv naming our scripts/artifacts/repo path, a PID we launched + recorded (§11.4.147), or a lock/port path under our tree — NEVER a loose `pgrep java/gradle/node` name match (it matches every project on the host). NEVER kill/signal/restart a process not positively ours (other projects' gradle/java/podman/zig are off-limits — operator-workload-safety, §12/§11.4.133); the §12.8 daemon-kill remediation MUST be scoped to OUR daemons. When our work waits on host contention from another project's process, surface it (§11.4.66/§11.4.101) — block-don't-break. Every `pgrep/ps` filter MUST exclude self-matches + other-project matches. Forensic anchor (FACT 2026-06-29): a 200%-CPU java+gradlew on the shared host was a separate `catalogizer` project's gradle test (cwd=Projects/catalogizer), nearly mis-attributed to our build. Composes §11.4.6/§11.4.66/§11.4.101/§11.4.147/§12/§12.8/§11.4.133. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-174-PROPAGATION` (literal `11.4.174`) + recommended gate `CM-PROCESS-OWNERSHIP-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.174. Non-compliance is a release blocker. No escape hatch — no `--assume-process-ours`, `--loose-pgrep-ok`, `--kill-any-matching-process`, `--skip-ownership-check` flag.
